

ROS2-李浩

1.什么是ROS2

ROS2为机器人程序的通信与组织框架。在rm中，opencv负责二值化，找轮廓，PnP，图像处理。ROS2负责把相机图像送给 Detector，把 Detector 的结果送给 Tracker，把 Tracker 的结果送给串口程序，管理参数，管理坐标系，启动整套程序。

ROS2核心设计为模块化，利用“node”(相当于节点) 解构系统

2.node之间的交流

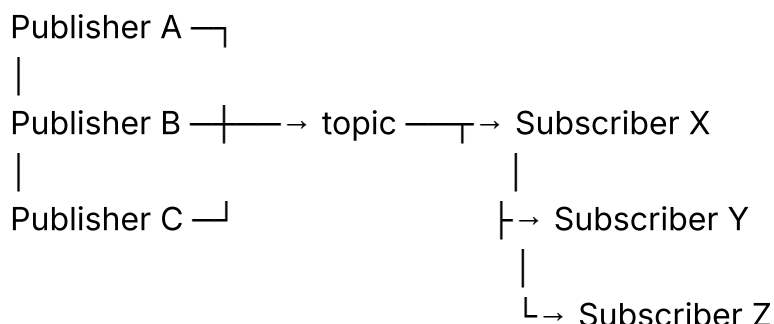
Topic为ROS2的节点交换消息的核心机制之一。采用Publish / Subscribe机制，一张图片进行交流时，把图片封装为图像消息，发布到某个 Topic，topic有 publish,subscribe，同一个节点可以发布多个 Topic，也可以订阅多个 Topic。

topic的作用是让单个node节点不用关心有几个订阅者，及其用途，订阅者不用关心topic来源。

topic有message概念：

message：用于 Topic 发布/订阅的数据类型。相当于cpp的struct，用于规定通信类型。

总之：topic是一个高效的数据通道。



3.ROS Graph

ROS Graph 就是运行中的 ROS 节点以及它们之间连接关系形成的网络，程序运行起来之后动态形成的。

Discovery机制让node可以发现彼此并建立链接。

tip: topic带来了解耦，在逻辑上不需要接待您知道彼此的来源，去处，但是在底层数据必须有接受发送的过程，这是discovery的事情。

4.Publisher / Subscriber 到底是什么

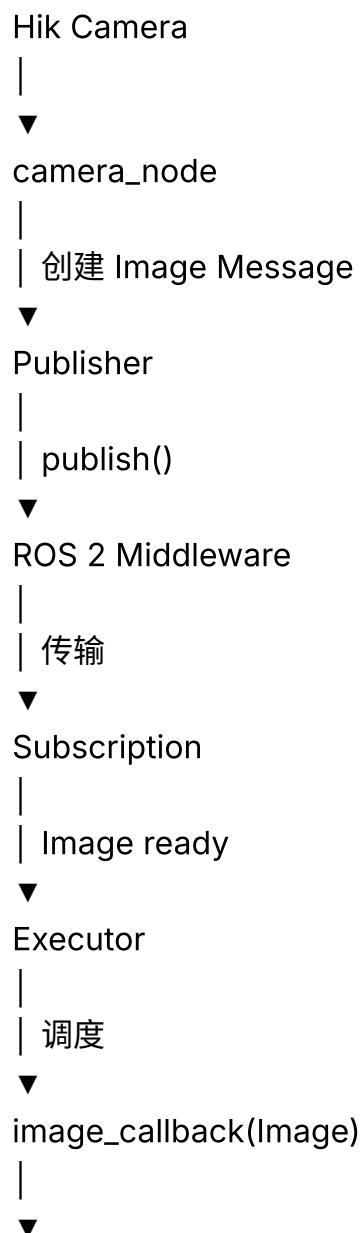
Publisher 和 Subscriber 其实是 Node 创建出来的**通信 Endpoint**。

Callback: 当某个事件发生时，希望 ROS 帮你执行的函数。

也就是事件驱动，当一个消息来时，执行对应操作。

Executor: 负责等待 ROS 事件，并决定什么时候执行 Callback 的调度器。

例如：



OpenCV

|



装甲板识别

5.Service

Service相当于ROS中的远程函数调用

topic更相当于广播，service更像询问。

ROS 2 的 Service 是典型的 **Client / Server** 模型。Client 发 Request，Server 处理后返回 Response。

Client

|

| Request

| a=10,b=20



Service Server

|

| Response

| sum=30



Client

Request 和 Response 是成对的。不像topic多对多。

一个服务可以有多个客户端，同名服务通常应只有一个服务端。

topic为

Publisher

Subscriber

service为

Client

Server

service也有自己的数据类型 (.srv)

	Topic	Service
模式	发布/订阅	请求/响应
通信方向	主要单向数据流	双向一问一答
是否持续	通常持续	通常一次调用
是否期待结果	不需要	需要
多接收者	很自然	不是主要用途
高频数据	很适合	不适合
典型用途	Camera、IMU	reset、配置、查询

发布状态优先用topic，请求，询问优先用service

调用一次，处理一次，返回一次

service也是事件驱动的，和executor有机结合

完整链路

Client

|

| Request

▼

Middleware

|

▼

Service Server

|

▼

Executor

|

▼

Callback

|

| 处理

▼

Response

|

▼

Middleware

|



Client

6.QOS

QoS 是配置通信行为的一组策略，包括消息保留、可靠性、持久性

核心概念:

History

Depth

Reliability

Durability

•History

有两个策略:

KEEP_LAST

KEEP_ALL

KEEP_LAST:保留最多一定数量的消息

KEEP_ALL:尽可能保留所有消息

•Depth

KEEP_LAST到底保留几条

对于History = KEEP_LAST，通常有Depth=n

意义：对于即时的传感器，操作，需要的是最新的信息，而过去的积压信息意义越来越小

•Reliability

主要策略:

RELIABLE

BEST_EFFORT

•RELIABLE：尽量把信息送达

•BEST_EFFORT：追求时延，不保证丢包率

•Durability

有两个策略:

VOLATILE

TRANSIENT_LOCAL

VOLATILE：Subscriber 只接收它加入之后的新消息。

TRANSIENT_LOCAL: Publisher 可以保留一定历史状态，让晚加入的 Subscriber 也能拿到。